ASD1-M1-14 · MINI-PROJEKT M1

Rekurencja krok po kroku

Zobacz wywołanie.

Przewidź powrót.

Edvin Zenevič · indeks 93783

Algorytmy i struktury danych I · 2026/2027

```
f(n)
↳ f(n−1)
← wynik
```

02 / Problem i cel

Nie tylko „funkcja wywołuje siebie”

- Każde wywołanie ma własne argumenty.
- Rodzic czeka na wynik dziecka.
- Baza zatrzymuje zejście, nie wszystkie powroty.

Odbiorca: student poznający rekurencję.

Stos ≠ drzewo

Stos: co jeszcze pracuje?
Drzewo: co zostało wywołane?

Cel: umieć przewidzieć następny krok na nowych danych.

03 / Zakres projektu

01

Suma przez podział

0–16 liczb · −20…20
pusty fragment = 0

02

Potęgowanie

Dwa warianty · a: −10…10
n: 0…10 · $0^0 = 1$

03

Wieże Hanoi

0–8 krążków · A → C
legalność każdego ruchu

Własne dane · 3 scenariusze/moduł · Krok · Start/Pauza · Reset · tempo · pytania

Widok stanu, argumentów, zmiennych, pseudokodu, stosu i drzewa.

04 / Algorytmy i mechanizmy

Suma

```
[2,4,1,3]
  ↙      ↘
[2,4]   [1,3]
  6   +   4
      10
```

Bazy: pusty / jeden element

Potęga szybka

`n = 5 → 2 → 1`

`half obliczamy RAZ`

`2 → 4 → 32`

Kwadrat + czynnik a
dla nieparzystego n

Hanoi

```
n−1: from → aux
n:   from → to
n−1: aux  → to
```

n = 0: bez ruchu

$M(n) = 2^n - 1$

Warunek zakończenia: każdy podproblem jest mniejszy.

05 / Struktury danych

Zdarzenie wykonania

`type · id · parentId · depth`
`args · line · reason`
`locals / value / move`

Unikatowe id wiąże ruch lub wynik
z konkretnym wywołaniem.

Stan odtwarzacza

- Tablica stosu → push / pop
- Map węzłów → wynik zostaje
- A, B, C → tablice od dołu do góry
- cursor → następne zdarzenie

Wejście: + ramka · Powrót: − ramka, + wynik w drzewie

depth korzenia = 0; maksymalny stos liczymy w ramkach, korzeń = 1.

06 / Technologie i architektura

Wejście + walidacja → Algorytm + zdarzenia → Odtwarzacz stanu → Widok + pytanie

Produkt

HTML + CSS + JavaScript
Bez bibliotek UI i API
Otwórz lokalny index.html

Moduły

algorithms.js → playback.js → ui.js
content.js: polskie objaśnienia

Weryfikacja

Node.js: algorytmy i stan
Playwright/Chromium: interfejs
Python/Chromium: dokumenty

> Algorytm nie zna DOM.
> Zegar nie zmienia wyniku.

07 / Realizacja i interfejs



Własny zrzut aplikacji: powrót #3 z wartością 2; na stosie pozostają #2 i #1.

01 · Obliczenia → zdarzenia

02 · Odtwarzanie → stan

03 · Stan → wyjaśnienie i pytanie

▲ szczyt · ● aktywne · ✓ zakończone · ▶ bieżący kod — nie tylko kolor.

08 / Testy i porównanie

44/44
testy domenowe PASS

17/17
kontrole UI PASS

462
przebiegi potęgowania

n	2^n	Mnożenia zwykłe	Mnożenia szybkie	Stos zwykły	Stos szybki
1	2	1	0	2	1
4	16	4	2	5	3
8	256	8	3	9	4
10	1024	10	4	11	4

Podstawa 2 · mnożenia = operacje * · stos = maks. liczba aktywnych ramek

Przypadki bazowe, walidacja, końcowy pusty stos i dwa jawne stany pośrednie.
Wyniki zapisane w wyniki/*.json; testy dostępne również w aplikacji.

09 / Wnioski i ograniczenia

Co pokazuje laboratorium?

- Suma: $\Theta(n)$, stos $\Theta(\log n)$
- Potęga zwykła: $\Theta(n + 1)$
- Potęga szybka: $\Theta(\log n)$, $n \geq 2$
- Hanoi: $\Theta(2^n)$, stos $\Theta(n + 1)$

Mały stos nie oznacza małej liczby ruchów.

Uczciwe granice

Małe dane; ślad w pamięci.
Walidator i renderowanie liczone osobno.
Brak benchmarku czasu ściennego.

Wkład i źródła

Edvin: zadanie, wymagania, dane autora.
AI (Codex/OpenClaw): kod i materiały.
Kontrola: testy + uruchomiony interfejs.

Źródła: karta ASD1-M1-14 · Princeton „Recursion” · kod i wyniki projektu. Szczegóły: raport, s. 10.

10 / Teraz: pokaz produktu

1. **Suma [2,4,1,3]** — wejście, baza, powrót, pytanie o stos
2. **Własne dane** — nowy przebieg; pusta tablica
3. **Potęga 2^5** — oba warianty, porównanie; 0^0
4. **Hanoi n = 3** — ruch ↔ wywołanie, cel C, 7 ruchów
5. **Testy i kod** — pokaż wyniki i wyjaśnij wskazaną funkcję

Zakończ slajdy. Otwórz index.html. Odpowiedz na pytania.